

Capstone studio and review

Week 10, Lecture 2 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- Worked example: tracing a complete agent from task string to validated output
- Adding reliability guards: step limits, failure caps, planner, task framing
- Measuring success with a task-set scorecard: categories, live pages, and what scores mean
- Presenting and defending the design: four questions you must be able to answer

Part 1: the worked example

The task: product-price monitor

```
"Go to the product page at x-product-url.  
If there are size or color variants, select the first available one.  
Extract the product name, current price in USD, and stock status.  
If the price is not visible after two navigation attempts,  
set price_usd to -1 and in_stock to False. Return the result."
```

What makes this agent-ready:

- Stopping condition: named explicitly
- Variant handling: explicit instruction + escape
- URL as sensitive_data: never in the LLM context
- One job: extraction only

Step-by-step trace

Step	Perceive	Decide	Act
1	URL in task, blank page	navigate to x-product-url	go to product page
2	DOM with variant dropdowns indexed 0-7	variant present, select first	click element 2
3	DOM with price visible, element 14	price found, extract	extract page content
4	content in memory	have name, price, stock status	done(output=ProductResult(...))

The selector map (week 2) gives the agent integer handles. AgentOutput (week 4) carries memory across steps so the name found in step 3 is not forgotten.

The output model

```
from pydantic import BaseModel, Field
from typing import Optional

class ProductResult(BaseModel):
    name: str = Field(description="Product name as shown on the page")
    price_usd: float = Field(description="Price in USD, no currency symbol")
    in_stock: bool = Field(description="True if add-to-cart is available")
    sku: Optional[str] = Field(None, description="SKU if shown, else None")
```

Validation happens before the result reaches calling code. A schema mismatch is a scored failure, not silent data corruption.

Part 2: reliability guards

The four guards

```
agent = Agent(  
    task="...",  
    llm=llm,  
    browser_session=session,  
    output_model=ProductResult,  
    max_steps=15,          # step ceiling  
    max_failures=3,        # consecutive failure cap  
    # planner=planner_llm  # for tasks with 15+ steps  
)
```

Each guard addresses a different failure mode:

- `max_steps`: prevents infinite loops on stagnant pages
- `max_failures`: stops early on broken task sequences
- Planner model: prevents drift in long tasks with many sub-goals

Task framing as the primary guard

No configuration parameter fixes a vague task string.

```
FRAGILE: "Find the price."
```

```
Agent loops on a variant page because "find the price" is  
achievable on any page it lands on.
```

```
RELIABLE: "Extract the price. If a variant selector is present,  
click the first available option first.  
If the price field is not visible after two clicks,  
set price_usd to -1 and call done."
```

An explicit escape routes the agent out of loops that configuration alone cannot break (browser-use contributors, 2026, service.py).

The procedural memory field

`AgentOutput.memory` (week 4) lets the agent carry notes across steps:

```
{  
  "evaluation_previous_goal": "Clicked first variant. Price now visible.",  
  "memory": "Product name: Blue Hiking Boot. Price: $89.99. In stock: true.",  
  "next_goal": "Call done with the extracted result.",  
  "action": [{"done": {"text": "..."}}]  
}
```

The agent does not re-read the product name from DOM every step. It writes it to memory and reads it back. This is why long tasks do not drift.

Part 3: measuring success

Why a single run is not a score

One run tells you the agent works on one input. A task-set scorecard tells you where and how often it fails across the input space. That is the information you need to improve it.

Build a task set of 10-20 inputs covering:

- The happy path (standard product page)
- Edge cases (variant selector, out-of-stock, login required)
- Expected failures (CAPTCHA, PDF-only price, broken page)

Each task has a known correct output you can check automatically.

Failure categories

Category	Meaning	Fix direction
pass	Output correct	None
fail-wrong-value	Output present, one field incorrect	Task framing or custom extraction
fail-no-output	Done called, schema validation failed	Output model field types
fail-loop	max_steps hit before done	max_steps too low, or escape missing
fail-crash	Unhandled exception	Code fix

Categorize each failure. The distribution tells you where to work next.

What scores on live pages mean

High scores on cached snapshots fall sharply on live websites. COLM 2025 (arXiv 2504.01382) showed this pattern consistently for web agents across benchmarks.

Three questions to ask before trusting a score:

1. Were the tasks run on live pages or cached HTML?
2. Were the tasks run at different times of day (price changes, stock changes)?
3. Do the tasks cover the failure-mode inputs, not just the happy path?

A score of 90% on five hand-picked happy-path tasks proves little.

Part 4: presenting the design

The four questions

Every capstone defense answers these in order:

1. **Scope:** what is the stopping condition, and why is this one agent rather than two?
2. **Tools:** for each custom action, what could the built-in not do? For each built-in kept, why was it sufficient?
3. **Schema:** name every field in the output model. Which are Optional and when are they None?
4. **Scorecard:** report at least one pass and one failure. For the failure: category, root cause, proposed fix.

A defense that only presents the happy path has not demonstrated understanding.

The agent-scorecard rubric

Dimension	What reviewers check
Task scoping	Stopping condition explicit; checklist passes
Tool choice	Justification for each built-in or custom decision
Output schema	Typed, flat, described, Optional used correctly
Scorecard	5+ tasks, categories named, failures analyzed
Defense	Four questions answered with specific evidence

A web agent whose score drops from 85% to 32% when tested on live websites is not a reliable web agent. It is a reliable cached-snapshot agent. The distinction matters for everything downstream.

Take-aways

1. Trace every step of your agent before the capstone defense. You should be able to say what the perceive, decide, and act phases produced at each step.
2. Reliability guards (`max_steps` , `max_failures`) cap damage; they do not fix bad task framing. Fix the task string first.
3. A task-set scorecard with failure categories is the minimum credible evidence that an agent works. Single-run demos are not.
4. The four defense questions (scope, tools, schema, scorecard) cover everything you built in ten weeks.

What's next

- **Section (this week):** Capstone studio: scope, build, run the task set, and defend your design in pairs
- **Capstone project due:** `/c/browser-use-26su/hw/capstone`
- **After the course:** the browser-use GitHub repository is active; the design philosophy essay (Žunič, 2026) reads differently after you have shipped a real agent